Modern LLM Architectures and Inference

Decoder-only models, decoding, KV caching, quantization, fine-tuning, and serving

Scope and approach

This document is a structured technical learning guide, written to connect foundational ideas to modern practice. It distinguishes established concepts from fast-moving research directions. It is not a systematic literature review and should be read alongside the original papers and documentation in the references.

How to use this guide

Read sequentially for a course-like progression. For each topic, focus on (1) the problem it solves, (2) the mechanism, (3) trade-offs, and (4) how to test whether it works in a real system.

1. The modern decoder-only Transformer

A common LLM architecture stacks causal Transformer blocks. Tokens are embedded, position information is added or applied through a positional method, and hidden states pass through attention and MLP sublayers. The final hidden state is projected to vocabulary logits; softmax defines next-token probabilities.

Typical components

- RMSNorm or LayerNorm: normalize activations to stabilize deep networks.
- RoPE or another position method: encodes relative/absolute position structure.
- Causal self-attention: allows each position to use only previous tokens and itself.
- SwiGLU-style MLP: transforms each token independently with gated nonlinear projections.
- Residual connections: add each sublayer output to its input.

Details differ across model families. Architecture names alone do not tell you the quality of training data, alignment, inference stack, or deployment reliability.

2. Generation and decoding

At each generation step, the model returns logits for the next token. Greedy decoding selects the highest-probability token. Sampling draws from a distribution; temperature rescales logits, top-k restricts candidates to the k highest scores, and nucleus (top-p) sampling keeps a smallest set whose cumulative probability reaches p. These settings change diversity and error patterns, not the model's underlying knowledge.

Stopping and structured output

Generation can stop at an end-of-sequence token, a configured stop string, a maximum token count, or a tool/control-flow transition. JSON mode or constrained decoding can improve syntax validity, but schema validity does not guarantee factual correctness. Applications should validate outputs and handle malformed or partial responses.

Reproducibility

Sampling can be nondeterministic depending on kernels, batching, serving stack, and provider behavior. Log model ID, tokenizer, system/developer prompts, decoding parameters, tools, and retrieval context when debugging.

3. KV cache and serving economics

During autoregressive generation, previous tokens' keys and values can be cached so the model does not recompute all earlier projections every step. The KV cache reduces repeated work but consumes memory proportional to context length, batch size, layers, key/value dimensions, and numeric precision.

Multi-query attention shares one key/value head across many query heads; grouped-query attention shares key/value heads among groups. These designs reduce KV-cache size and can improve decode throughput, with quality trade-offs depending on training and model design. Continuous batching, paged KV-cache management, prefix caching, speculative decoding, and optimized kernels are serving-system techniques used to improve throughput or latency.

Useful metrics

- Time to first token (TTFT): perceived delay before the first generated token.
- Inter-token latency: delay between subsequent output tokens.
- Tokens per second: throughput per request or across the service.
- Concurrent request capacity: workload supported under latency targets.
- Cost per successful task: includes retries, retrieval, tool calls, and failed outputs.

4. Quantization and efficient adaptation

Quantization stores weights and/or activations at lower precision, such as 8-bit or 4-bit formats, to reduce memory and sometimes improve speed. Quality depends on model, quantization method, calibration data, hardware, and kernels. A quantized model that fits into memory may still be too slow or inaccurate for a given task.

LoRA adds trainable low-rank update matrices to selected weight matrices while freezing the base model. QLoRA combines low-bit base weights with trainable low-rank adapters, reducing fine-tuning memory. Adapters are useful when a model needs a task-specific behavior or domain format; they are not a reliable way to inject frequently changing factual knowledge.

Choosing a strategy

- Prompting: lowest setup cost; good for instruction and formatting changes.
- RAG: supplies external, updateable evidence at query time.
- LoRA/QLoRA: changes learned behavior or domain adaptation with lower training cost.
- Full fine-tuning: broad parameter adaptation, typically more compute and operational complexity.
- Distillation: trains a smaller model to approximate a larger model or teacher behavior.

5. Open-weight and hosted models

Hosted APIs simplify deployment and may offer strong performance, managed scaling, and integrated tools, but introduce provider dependence, usage cost, data-handling constraints, and version changes. Open-weight models offer more control over inference, fine-tuning, and deployment, but require hardware, security, updates, evaluation, and operational expertise.

Choose based on measured task quality, latency, context needs, privacy requirements, licensing, languages, tool use, reliability, and total cost. Do not assume that “open source,” “open weights,” and “free for every use” mean the same thing; review the actual license and terms.

Deployment checklist

- Pin versions and retain reproducible prompts/configuration.
- Load-test realistic concurrency and context lengths.
- Measure quality before and after quantization or adapter merging.
- Set rate limits, authentication, request-size limits, and abuse monitoring.
- Record model limitations, data handling, retention, and rollback procedures.

6. Common misconceptions

- A larger context window does not guarantee reliable use of every token.
- More parameters do not automatically mean lower hallucination rates.
- Fine-tuning is not a substitute for retrieval when facts change frequently.
- Quantization is not always lossless; test on the actual workload.
- A model's benchmark score does not guarantee performance in your product.
- An LLM response should be treated as a generated proposal until checked where correctness matters.

Suggested learning path and exercises

- Draw the path from token IDs to logits and next-token sampling.
- Estimate KV-cache memory for a model using layer count, KV heads, head dimension, sequence length, batch size, and bytes per value.
- Run a small local model in full precision and quantized precision; compare latency, memory, and task quality.
- Compare prompting, RAG, LoRA, and full fine-tuning for one concrete use case.
- Build a latency report with TTFT, inter-token latency, throughput, and cost per successful answer.

References and further reading

The references below are starting points. Prefer original papers, official documentation, and careful technical reports. Some papers are historical foundations; others describe techniques whose current implementations may have evolved.

Vaswani et al. (2017), Attention Is All You Need. https://arxiv.org/abs/1706.03762

Shazeer (2019), Fast Transformer Decoding: One Write-Head is All You Need. https://arxiv.org/abs/1911.02150

Ainslie et al. (2023), GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. https://arxiv.org/abs/2305.13245

Dao et al. (2022), FlashAttention. https://arxiv.org/abs/2205.14135

Dettmers et al. (2023), QLoRA: Efficient Finetuning of Quantized LLMs. https://arxiv.org/abs/2305.14314

Hu et al. (2021), LoRA: Low-Rank Adaptation of Large Language Models. https://arxiv.org/abs/2106.09685

Frantar et al. (2022), GPTQ. https://arxiv.org/abs/2210.17323

Xiao et al. (2023), SmoothQuant. https://arxiv.org/abs/2211.10438

Dubey et al. (2024), The Llama 3 Herd of Models. https://arxiv.org/abs/2407.21783